```cpp
#include <vector>

using ll = long long;
using ull = unsigned long long;

template <class T, class cmp = std::less<T>>
struct discretize {
    std::vector<T> vec;
    void build() {
        sort(vec.begin(), vec.end(), cmp());
        vec.resize(unique(vec.begin(), vec.end()) - vec.begin());
    }
    size_t operator[](const T &x) {
        return lower_bound(vec.begin(), vec.end(), x, cmp()) - vec.begin();
    }
};
```

```cpp
#include <array>
#include <string>
#include <vector>

namespace string_utils {
    using std::string;

    const std::vector<char> charset{'0', '1'};
    const size_t charset_size = 2;

    const size_t npos = static_cast<size_t>(-1);

    std::vector<size_t> calculate_nxt(const string &s) {
        std::vector<size_t> nxt(s.size() + 1);
        nxt[0] = npos;
        size_t nxt_pos = nxt[0];
        for (size_t i = 0; i < s.size(); i++) {
            while (nxt_pos != npos && s[nxt_pos] != s[i]) {
                nxt_pos = nxt[nxt_pos];
            }
            nxt_pos++;
            nxt[i + 1] = nxt_pos;
        }
        return nxt;
    }

    std::vector<std::array<size_t, charset_size>>
    kmp_automaton(const string &s, const std::vector<size_t> &nxt) {
        std::vector<std::array<size_t, charset_size>> res(s.size());
        for (size_t i = 0; i < s.size(); i++) {
            for (size_t j = 0; j < charset.size(); j++) {
                auto nxt_char = charset[j];
                if (nxt_char == s[i]) {
                    res[i][j] = i + 1;
                }
                else {
                    if (nxt[i] == npos) {
                        res[i][j] = 0;
                    }
                    else {
                        res[i][j] = res[nxt[i]][j];
                    }
                }
            }
        }
        return res;
    }

    std::vector<size_t> match(const string &s, const string &t,
                              const std::vector<size_t> &nxt) {
        std::vector<size_t> ret(t.size() + 1);
        size_t nxt_pos = 0;
        for (size_t i = 0; i < t.size(); i++) {
            while (nxt_pos == s.size() ||
                   (nxt_pos != npos && s[nxt_pos] != t[i])) {
                nxt_pos = nxt[nxt_pos];
            }
            nxt_pos++;
            ret[i + 1] = nxt_pos;
        }
        return ret;
    }
```



```cpp
}; // namespace string_utils
```

```cpp
#include <vector>

namespace ds {
using std::vector;

template <class T>
struct adj_table {

    struct node {
        T val;
        size_t nxt;
    };

    static const size_t npos = static_cast<size_t>(-1);

    vector<size_t> head;
    vector<node> nd;

    adj_table(size_t n) : head(n, npos), nd() {}

    adj_table() {}

    void insert(size_t pos, const T &_val) {
        nd.push_back(node{_val, head[pos]});
        head[pos] = nd.size() - 1;
    }

    size_t next_pos(size_t i) { return nd[i].nxt; }
};

template <class T>
const size_t adj_table<T>::npos;
} // namespace ds
```

2026–03–20 19:41 **bit.hpp** é¡µ 1/1

```cpp
#include <array>
#include <vector>

using namespace std;

using ll = long long;
using ull = unsigned long long;

size_t lowbit(size_t x) { return x & -x; }

template <class T, size_t N>
struct bit {
    array<T, N> t;
    void add(size_t p, T val) {
        for (; p < N; p += lowbit(p)) {
            t[p] += val;
        }
        return;
    }
    T query(size_t p) const {
        if (p < 0) {
            return 0;
        }
        T ret = 0;
        for (; p; p -= lowbit(p)) {
            ret += t[p];
        }
        return ret;
    }
    T query(size_t l, size_t r) const { return query(r) - query(l - 1); }
};

template <class T, size_t N>
struct bit_with_coef {
  private:
    void add(size_t pos, T val) {
        // è¿M–^Yä¸ªåM–^G½æM–^U°è¿M–^TåM–^[M–^^çM–^ZM–^DæM–^X¯åM–^IM–^Mç¼M–^@å
M–^RM–^LãM–^@M–^B
        // å¹¶ä¸M–^MæM–^X¯åM–^MM–^UçM–^B¹çM–^ZM–^DåM–^@¼ãM–^@M–^B
        b.add(pos, val);
        c.add(pos, val * sum_coef[pos - 1]);
        return;
    }
    ll query(size_t r) const { return sum_coef[r] * b.query(r) - c.query(r); }

  public:
    array<T, N> sum_coef;
    bit<T, N> b, c;
    void build(const vector<T> &vec) {
        copy(vec.begin(), vec.end(), sum_coef.begin());
        for (size_t i = 1; i < vec.size(); i++) {
            sum_coef[i] += sum_coef[i - 1];
        }
        return;
    }
    void add(size_t l, size_t r, T val) {
        add(l, val);
        add(r + 1, -val);
        return;
    }
    ll query(size_t l, size_t r) const { return query(r) - query(l - 1); }
};
```

2026−03−20 19:41 **dsu.hpp** é¡µ 1/1

```cpp
#include <vector>
using namespace std;

using ll = long long;
using ull = unsigned long long;

namespace data_structure {
struct dsu {
    vector<size_t> fa;
    dsu() {}
    void push_back() { fa.push_back(fa.size()); }
    void resize(size_t sz) {
        while (fa.size() < sz) {
            push_back();
        }
    }
    size_t find_root(size_t p) {
        if (fa[p] == p) {
            return p;
        }
        else {
            return fa[p] = find_root(fa[p]);
        }
    }
    bool is_same(size_t x, size_t y) { return find_root(x) == find_root(y); }
    void merge(size_t x, size_t y) {
        size_t rx = find_root(x), ry = find_root(y);
        if (rx != ry) {
            fa[rx] = ry;
        }
        return;
    }
};
} // namespace data_structure
```

```cpp
#include "math/modular.hpp"

namespace string_utils {
template <ull base, ull mod>
struct hash_string {
    std::vector<maths::modular<mod>> prime_pow, h;
    std::string s;
    static const size_t npos = -1;
    hash_string(const std::string &_s) : s(_s) {
        prime_pow.resize(s.size() + 1);
        prime_pow[0] = 1;
        for (size_t i = 1; i < prime_pow.size(); i++) {
            prime_pow[i] = base * prime_pow[i - 1];
        }
        h.resize(s.size() + 1);
        h[0] = 0;
        for (size_t i = 1; i < s.size(); i++) {
            h[i] = h[i - 1] * base + s[i - 1] - 'A';
        }
        return;
    }

    maths::modular<mod> substr(size_t pos, size_t len) {
        return h[pos + len] - h[pos] * prime_pow[len];
    }

    size_t size() const { return s.size(); }

    size_t find(const hash_string &str, size_t pos = 0) {
        for (size_t i = pos; i + str.size() <= s.size(); i++) {
            if (substr(i, str.size()) == str.h.back()) {
                return pos;
            }
        }
        return npos;
    }
};
} // namespace string_utils
```

```cpp
#include <functional>
#include <vector>

namespace data_structure {
template <class info_t, class tag_t, class merge_info_op = std::plus<info_t>,
          class apply_op = std::multiplies<void>,
          class merge_tag_op = std::plus<tag_t>>
class lazy_segtree {
  public:
    const merge_info_op merge_info;
    const apply_op apply_to_info;
    const merge_tag_op merge_tag;
    const info_t id_info;
    const tag_t id_tag;
    const size_t npos = static_cast<size_t>(-1);

    struct node {
        info_t info;
        tag_t tag;

        size_t l, r;

        size_t mid() const { return (l + r) / 2; }
    };

    std::vector<node> t;

  private:
    size_t lc(const size_t &p) const { return 2 * p; }
    size_t rc(const size_t &p) const { return 2 * p + 1; }
    void update(size_t p) {
        t[p].info = merge_info(t[lc(p)].info, t[rc(p)].info);
        return;
    }

    void apply_to_node(size_t p, const tag_t &tag) {
        t[p].tag = merge_tag(tag, t[p].tag);
        t[p].info = apply_to_info(tag, t[p].info);
    }

    void spread(size_t p) {
        apply_to_node(lc(p), t[p].tag);
        apply_to_node(rc(p), t[p].tag);
        t[p].tag = id_tag;
        return;
    }

    void build(size_t l, size_t r, const std::vector<info_t> &init_val,
               size_t p = 1) {
        t[p].l = l;
        t[p].r = r;
        t[p].info = id_info;
        t[p].tag = id_tag;
        if (l + 1 == r) {
            t[p].info = init_val[l];
            return;
        }
        size_t mid = t[p].mid();
        build(l, mid, init_val, lc(p));
        build(mid, r, init_val, rc(p));
        update(p);
        return;
```



```cpp
    }

  public:
    lazy_segtree()
        : merge_info(), apply_to_info(), merge_tag(), id_info(), id_tag(), t() {
    }

    lazy_segtree(size_t n, const merge_info_op &_merge_info = merge_info_op(),
                 const apply_op &_apply = apply_op(),
                 const merge_tag_op &_merge_tag = merge_tag_op(),
                 const info_t &_info_id = info_t(),
                 const tag_t &_tag_id = tag_t())
        : merge_info(_merge_info), apply_to_info(_apply), merge_tag(_merge_tag),
          id_info(_info_id), id_tag(_tag_id), t(n * 4) {
        build(0, n, std::vector<info_t>(n, id_info));
    }

    template <class iter>
    void assign(iter iter_begin, iter iter_end) {
        size_t n = std::distance(iter_begin, iter_end);
        build(0, n, std::vector<info_t>(iter_begin, iter_end));
    }

    template <class iter>
    lazy_segtree(iter iter_begin, iter iter_end,
                 const merge_info_op &_merge_info = merge_info_op(),
                 const apply_op &_apply = apply_op(),
                 const merge_tag_op &_merge_tag = merge_tag_op(),
                 const info_t &_info_id = info_t(),
                 const tag_t &_tag_id = tag_t())
        : merge_info(_merge_info), apply_to_info(_apply), merge_tag(_merge_tag),
          id_info(_info_id), id_tag(_tag_id),
          t(std::distance(iter_begin, iter_end) * 4) {
        assign(iter_begin, iter_end);
    }

    void apply(size_t l, size_t r, const tag_t &tag, size_t p = 1) {
        if (l <= t[p].l && t[p].r <= r) {
            apply_to_node(p, tag);
            return;
        }
        size_t mid = t[p].mid();
        spread(p);
        if (l < mid) {
            apply(l, r, tag, lc(p));
        }
        if (mid < r) {
            apply(l, r, tag, rc(p));
        }
        update(p);
        return;
    }

    info_t query(size_t l, size_t r, size_t p = 1) {
        if (l <= t[p].l && t[p].r <= r) {
            return t[p].info;
        }
        size_t mid = t[p].mid();
        spread(p);
        info_t ret = id_info;
        if (l < mid) {
            ret = merge_info(query(l, r, lc(p)), ret);
```

```
        }
        if (mid < r) {
            ret = merge_info(ret, query(l, r, rc(p)));
        }
        update(p);
        return ret;
    }

    void set(size_t pos, const info_t &info, size_t p = 1) {
        if (t[p].l + 1 == t[p].r) {
            t[p].info = info;
            return;
        }
        size_t mid = t[p].mid();
        spread(p);
        if (pos < mid) {
            set(pos, info, lc(p));
        }
        else {
            set(pos, info, rc(p));
        }
        update(p);
        return;
    }
};
} // namespace data_structure
```

```cpp
#include <algorithm>
#include <functional>
#include <vector>

using namespace std;

using ll = long long;
using ull = unsigned long long;

namespace data_structure {
namespace functor {
template <class T, class cmp = less<T>>
struct min_element {
    cmp compare;
    min_element() : compare() {}
    min_element(cmp c) : compare(c) {}
    T operator()(const T &a, const T &b) const {
        return std::min(a, b, compare);
    }
};

template <class T, class cmp = less<T>>
struct max_element {
    cmp compare;
    max_element() : compare() {}
    max_element(cmp c) : compare(c) {}
    T operator()(const T &a, const T &b) const {
        return std::max(a, b, compare);
    }
};
} // namespace functor

template <class T, class binary_op = functor::min_element<T>>
struct sparse_table {
    vector<vector<T>> vec;
    binary_op op;
    sparse_table() : vec(), op() {}
    template <class iter>
    sparse_table(iter begin, iter end, binary_op _op = binary_op()) : op(_op) {
        size_t sz = distance(begin, end);
        size_t lg_sz = __lg(sz) + 1;
        vec.resize(lg_sz + 1);
        for (size_t i = 0; i < lg_sz; i++) {
            vec[i].resize(sz);
        }
        copy(begin, end, vec[0].begin());
        for (size_t i = 1; i < lg_sz; i++) {
            for (size_t j = 0; j < sz; j++) {
                size_t nxt = j + (1 << (i - 1));
                if (nxt >= sz) {
                    vec[i][j] = vec[i - 1][j];
                }
                else {
                    vec[i][j] = op(vec[i - 1][j], vec[i - 1][nxt]);
                }
            }
        }
    }
    T query(size_t l, size_t r) const {
        size_t sz = r - l;
        size_t lg = __lg(sz);
        return op(vec[lg][l], vec[lg][r - (1 << lg)]);
```



```cpp
    }
};
} // namespace data_structure
```

2026−03−20 19:41 **sum_prefix.hpp** é¡µ 1/1

```cpp
#include <functional>
#include <vector>

using namespace std;

using ll = long long;
using ull = unsigned long long;

namespace data_structure {
template <class T, class binary_op = plus<T>, class inv_op = minus<T>>
struct sum_prefix {
    const size_t npos = -1;
    vector<T> vec;
    const T unit_element;
    sum_prefix(const vector<T> &_vec, T _unit = T()) : unit_element(_unit) {
        vec = _vec;
        for (size_t i = 0; i < vec.size() - 1; i++) {
            vec[i + 1] += vec[i];
        }
        return;
    }
    template <class iter>
    sum_prefix(iter begin, iter end, T _unit = T()) : unit_element(_unit) {
        vec.resize(distance(begin, end));
        copy(begin, end, vec.begin());
        for (size_t i = 0; i < vec.size() - 1; i++) {
            vec[i + 1] = binary_op()(vec[i + 1], vec[i]);
        }
    }
    T query(size_t l, size_t r) const {
        return inv_op()(vec[r - 1], (l == 0 ? unit_element : vec[l - 1]));
    }
    T query(size_t pos) {
        return pos - 1 == npos ? unit_element : vec[pos - 1];
    }
};
} // namespace data_structure
```

```cpp
#include "data_structure/adj_table.hpp"
#include "math/numbers.hpp"
#include <queue>
#include <vector>

namespace graph {
template <typename result_type>
struct network {
    struct edge {
        size_t src, dest;
        result_type vol;
    };

    ds::adj_table<edge> g;

    size_t s, t;
    size_t n;

    std::vector<uint> dis;
    std::vector<size_t> cur;

    bool bfs() {
        std::fill(dis.begin(), dis.end(), numbers::inf<uint>());
        std::vector<bool> vis(n);

        std::queue<size_t> q;

        q.push(s);
        vis[s] = true;
        dis[s] = 0;

        while (!q.empty()) {
            auto f = q.front();
            q.pop();

            for (size_t i = g.head[f]; i != ds::adj_table<edge>::npos;
                 i = g.next_pos(i)) {
                const edge &e = g.nd[i].val;
                if (!vis[e.dest] && e.vol != 0) {
                    dis[e.dest] = dis[f] + 1;
                    vis[e.dest] = true;
                    q.push(e.dest);
                }
            }
        }

        cur = g.head;

        return dis[t] != numbers::inf<uint>();
    }

    result_type dfs(size_t p, result_type max_flow) {
        if (p == t) {
            return max_flow;
        }

        result_type final_flow = 0;
        for (; cur[p] != ds::adj_table<edge>::npos;
             cur[p] = g.next_pos(cur[p])) {
            const auto &i = cur[p];
            const edge &e = g.nd[i].val;
```



```cpp
            if (dis[e.dest] != dis[p] + 1) {
                continue;
            }

            auto r = dfs(e.dest, std::min(e.vol, max_flow));
            g.nd[i].val.vol -= r;
            g.nd[i ^ 1].val.vol += r;
            final_flow += r;
            max_flow -= r;

            if (!max_flow) {
                /**
                 * çM–^T±ä°M–^N c++ ä¼M–^ZåM–^EM–^Hè¿M–^[è¡M–^L cur[p] = g.next_pos(cur[p]) è¿M–^Yä¸ªè¯–åM–^O¥åM–^FM–^MåM–^H¤æM–^V–åM–^PM–^Hæ³M–^Uä¸M–^NåM–^P¦ï¼M–^L
                 * åM–^[ æ–¤å¦M–^BæM–^^M–^\ max_flow æM–^A°å¥½åM–^GM–^OåM–^H° 0 å®M–^^éM–^YM–^Eä¸M–^Jè¿M–^YéM–^GM–^Lå¹¶æ²¡æM–^\M–^IæµM–^Aæ»¡ãM–^@M–^B
                 */
                break;
            }
        }

        return final_flow;
    }

    result_type dinic() {
        result_type r = 0;
        while (bfs()) {
            r += dfs(s, numbers::inf<result_type>());
        }
        return r;
    }

    network(size_t _n, size_t _s, size_t _t)
        : g(_n), s(_s), t(_t), n(_n), dis(_n), cur(_n) {}

    void add_edge(size_t src, size_t dest, result_type vol) {
        g.insert(src, {src, dest, vol});
        g.insert(dest, {dest, src, 0});
    }
};

} // namespace graph
```

```cpp
#include <queue>
#include <vector>

namespace graph {
template <typename result_type, typename cost_type>
struct network_with_cost {
    struct edge {
        size_t src, dest;
        result_type vol;
        cost_type cost;
    };

    ds::adj_table<edge> g;

    size_t s, t;
    size_t n;

    std::vector<cost_type> dis;
    std::vector<size_t> prev_node;

    constexpr static const size_t npos = static_cast<size_t>(-1);

    bool spfa() {
        std::fill(dis.begin(), dis.end(), numbers::inf<uint>());
        std::fill(prev_node.begin(), prev_node.end(),
                  network_with_cost<result_type, cost_type>::npos);

        std::queue<size_t> q;

        q.push(s);
        dis[s] = 0;

        while (!q.empty()) {
            auto f = q.front();
            q.pop();

            for (size_t i = g.head[f]; i != ds::adj_table<edge>::npos;
                 i = g.next_pos(i)) {
                const edge &e = g.nd[i].val;
                if (dis[e.dest] > dis[f] + e.cost && e.vol != 0) {
                    dis[e.dest] = dis[f] + e.cost;
                    prev_node[e.dest] = i;
                    q.push(e.dest);
                }
            }
        }

        return dis[t] != numbers::inf<cost_type>();
    }

    struct flow {
        result_type vol;
        cost_type cost;
    };

    flow ek() {
        flow ret{0, 0};
        while (spfa()) {
            result_type r = numbers::inf<result_type>();
            cost_type c = 0;
            {
                size_t cur = t;
```



```cpp
                while (cur != s) {
                    const auto &e = g.nd[prev_node[cur]].val;
                    c += e.cost;
                    r = std::min(r, e.vol);
                    cur = e.src;
                }
            }
            ret.cost += r * c;
            ret.vol += r;
            {
                size_t cur = t;
                while (cur != s) {
                    const auto &e = g.nd[prev_node[cur]].val;
                    g.nd[prev_node[cur]].val.vol -= r;
                    g.nd[prev_node[cur] ^ 1].val.vol += r;
                    cur = e.src;
                }
            }
        }
        return ret;
    }

    network_with_cost(size_t _n, size_t _s, size_t _t)
        : g(_n), s(_s), t(_t), n(_n), dis(_n), prev_node(_n) {}

    void add_edge(size_t src, size_t dest, result_type vol, cost_type cost) {
        g.insert(src, {src, dest, vol, cost});
        g.insert(dest, {dest, src, 0, -cost});
    }
};

template <typename result_type, typename cost_type>
const size_t network_with_cost<result_type, cost_type>::npos;
} // namespace graph
```

```cpp
#include <cassert>
using namespace std;

using ll = long long;
using ull = unsigned long long;

namespace maths {
struct dynamic_modular {
    ll x;
    ull mod;
    dynamic_modular() : x(0), mod(0) {}
    dynamic_modular(ll _x, ull _mod) : x(_x), mod(_mod) {
        x %= (ll)mod;
        if (x < 0) {
            x += mod;
        }
    }
    dynamic_modular operator+(const dynamic_modular &b) const {
        assert(mod == b.mod);
        return dynamic_modular(x + b.x, mod);
    }
    template <class T>
    dynamic_modular operator+(const T &b) const {
        return dynamic_modular(x + b, mod);
    }
    dynamic_modular operator*(const dynamic_modular &b) {
        assert(mod == b.mod);
        return dynamic_modular(x * b.x, mod);
    }
    template <class T>
    dynamic_modular operator*(const T &b) const {
        return dynamic_modular(x * b, mod);
    }
    dynamic_modular operator-(const dynamic_modular &b) const {
        assert(mod == b.mod);
        return dynamic_modular(x - b.x, mod);
    }
    template <class T>
    dynamic_modular operator-(const T &b) const {
        return dynamic_modular(x - b, mod);
    }
    template <class T>
    dynamic_modular operator+=(const T &b) {
        return *this = *this + b;
    }
    template <class T>
    dynamic_modular operator*=(const T &b) {
        return *this = *this * b;
    }
    template <class T>
    dynamic_modular operator-=(const T &b) {
        return *this = *this - b;
    }
    void setmod(ull _mod) {
        mod = _mod;
        x %= mod;
        if (x < 0) {
            x += mod;
        }
    }
};
```



```cpp
} // namespace maths
```

```cpp
#include <cmath>

using ld = long double;

namespace maths {
ld sq(ld x) { return x * x; }

struct point {
    ld x, y;
    point operator+(const point &b) const { return {x + b.x, y + b.y}; }
    point operator-(const point &b) const { return {x - b.x, y - b.y}; }
    point operator*(const ld &k) const { return {x * k, y * k}; }
    point operator/(const ld &k) const { return {x / k, y / k}; }
    point operator/=(const ld &k) { return *this = *this / k; }
    ld len() { return sqrt(sq(x) + sq(y)); }
};

ld dis(const point &a, const point &b) {
    return sqrt(sq(a.x - b.x) + sq(a.y - b.y));
}
} // namespace maths
```

```cpp
#include <array>
#include <functional>

namespace maths {
template <size_t N, size_t M, typename T = int,
          typename plus_op_t = std::plus<>,
          typename multiply_op_t = std::multiplies<>>
struct matrix {
    const plus_op_t plus_op;
    const multiply_op_t multiply_op;
    const T id_plus, id_multiplies;

    std::array<std::array<T, M>, N> mat;

    matrix clear() {
        for (size_t i = 0; i < N; i++) {
            mat[i].fill(id_plus);
        }
        return *this;
    }

    matrix reset() {
        clear();
        for (size_t i = 0; i < std::min(N, M); i++) {
            mat[i][i] = id_multiplies;
        }
        return *this;
    }

    matrix(const plus_op_t _plus_op = plus_op_t(),
           const multiply_op_t _multiply_op = multiply_op_t(),
           const T _id_plus = T(), const T _id_multiplies = T(1))
        : plus_op(_plus_op), multiply_op(_multiply_op), id_plus(_id_plus),
          id_multiplies(_id_multiplies) {
        clear();
    }

    matrix(const std::array<std::array<T, N>, M> &_mat,
           const plus_op_t _plus_op = plus_op_t(),
           const multiply_op_t _multiply_op = multiply_op_t(),
           const T _id_plus = T(), const T _id_multiplies = T(1))
        : plus_op(_plus_op), multiply_op(_multiply_op), id_plus(_id_plus),
          id_multiplies(_id_multiplies), mat(_mat) {}

    template <size_t K>
    friend matrix<K, M, T> operator*(const matrix<K, N, T> &lhs,
                                     const matrix<N, M, T> &rhs) {
        matrix<K, M, T, plus_op_t, multiply_op_t> ret(
            rhs.plus_op, rhs.multiply_op, rhs.id_plus, rhs.id_multiplies);
        ret.clear();
        for (size_t i = 0; i < N; i++) {
            for (size_t k = 0; k < K; k++) {
                for (size_t j = 0; j < M; j++) {
                    ret.mat[k][j] = ret.plus_op(
                        ret.mat[k][j],
                        ret.multiply_op(lhs.mat[k][i], rhs.mat[i][j]));
                }
            }
        }
        return ret;
    }
```



```cpp
    friend matrix operator+(const matrix &lhs, const matrix &rhs) {
        matrix ret(rhs.plus_op, rhs.multiply_op, rhs.id_plus,
                   rhs.id_multiplies);
        ret.clear();
        for (size_t i = 0; i < N; i++) {
            for (size_t j = 0; j < M; j++) {
                ret.mat[i][j] = ret.plus_op(lhs.mat[i][j], rhs.mat[i][j]);
            }
        }
        return ret;
    }

    matrix &operator=(const matrix &rhs) {
        mat = rhs.mat;
        return *this;
    }
};

} // namespace maths
```

2026–03–20 19:41 **modular.hpp** é¡µ 1/1

```cpp
#include <iostream>

using ll = long long;
using ull = unsigned long long;

namespace maths {
template <ull mod, class int_type = ll, class uint_type = ull>
class modular {
  private:
    uint_type x;
    void norm() { x -= mod * (x >= mod); }

  public:
    modular() : x(0) {}
    modular(int_type _x) {
        if (x < 0) {
            x = _x % (int_type)mod + (int_type)mod;
        }
        else {
            x = _x % mod;
        }
        norm();
        return;
    }
    friend modular operator+(const modular &lhs, const modular &rhs) {
        modular ret;
        ret.x = lhs.x + rhs.x;
        ret.norm();
        return ret;
    }
    friend modular operator-(const modular &lhs, const modular &rhs) {
        modular ret;
        ret.x = lhs.x + mod - rhs.x;
        ret.norm();
        return ret;
    }
    friend modular operator*(const modular &lhs, const modular &rhs) {
        return modular(lhs.x * rhs.x);
    }
    modular operator-() const {
        modular ret;
        ret.x = mod - x;
        return ret;
    }
    modular operator-=(const modular &b) { return *this = *this - b; }
    modular operator+=(const modular &b) { return *this = *this + b; }
    modular operator*=(const modular &b) { return *this = *this * b; }
    bool operator==(const modular &b) const { return x == b.x; }
    uint_type val() const { return x; }
    friend std::istream &operator>>(std::istream &is, modular &rhs) {
        is >> rhs.x;
        rhs.x %= mod;
        return is;
    }
    friend std::ostream &operator<<(std::ostream &os, const modular &rhs) {
        os << rhs.val();
        return os;
    }
};
} // namespace maths
```

```cpp
using uint = unsigned int;
using ll = long long;
using ull = unsigned long long;

namespace numbers {
template <typename T>
T inf() {
    assert(false);
}

template <>
int inf<int>() {
    return 0x3f3f3f3f;
}

template <>
uint inf<uint>() {
    return 0x3f3f3f3f;
}

template <>
ll inf<ll>() {
    return 0x3f3f3f3f3f3f3f3f;
}

template <>
ull inf<ull>() {
    return 0x3f3f3f3f3f3f3f3f;
}
} // namespace numbers
```

```cpp
#include <string>

using ll = long long;
using ull = unsigned long long;

namespace maths {
template <class T>
T quick_pow(T a, ull b, T id = T()) {
    T ret = id;
    for (; b; b >>= 1, a = a * a) {
        if (b & 1) {
            ret = a * ret;
        }
    }
    return ret;
}

template <class T>
T quick_pow(T a, const std::string &s, T id = T()) {
    T ret = id;
    for (size_t i = 0; i < s.size(); i++, a = a * a) {
        if (s[i] == '1') {
            ret = a * ret;
        }
    }
    return ret;
}

} // namespace maths
```